

Penetration Testing Report – OWASP Juice Shop

Tested by: Saloni Kumari

Date: 08 May 2026

Target: http://localhost:3000

1. Executive Summary

A penetration test was conducted on OWASP Juice Shop, an intentionally vulnerable web application. Multiple vulnerabilities were identified and successfully exploited including SQL injection, Cross-Site Scripting (XSS), exposed sensitive directories, and authentication bypass. This report documents the findings, exploitation steps, and remediation recommendations.

2. Scope

- In-scope: OWASP Juice Shop running on localhost port 3000
- Out-of-scope: Any other systems or real-world websites

3. Methodology

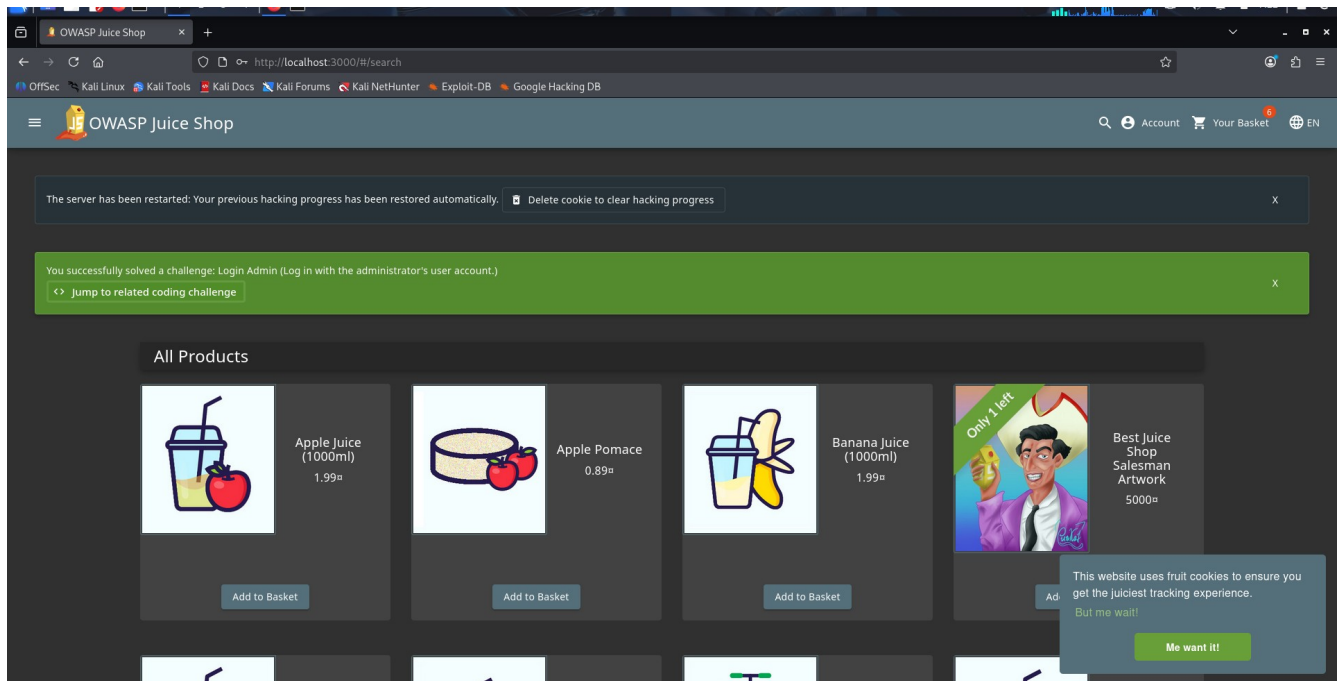
The following tools and techniques were used:

- SQL injection for login bypass
- Manual JavaScript payload injection for XSS
- Dirsearch for hidden directory discovery
- Burp Suite and curl for request manipulation (attempt)
- Browser testing for FTP directory exposure

4. Findings & Exploitation

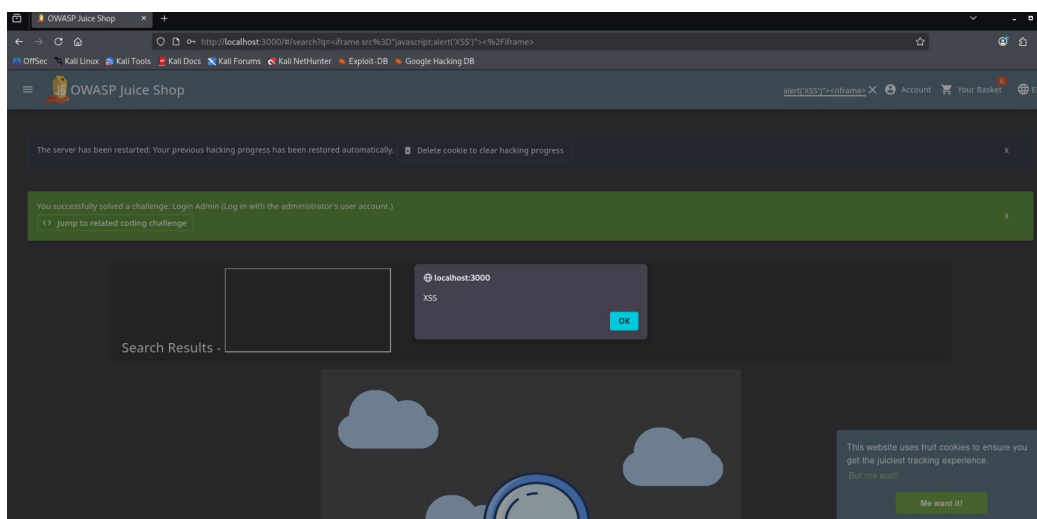
4.1 SQL Injection – Login Bypass

- Payload: admin' OR '1'='1' --
- Impact: Attacker can log in as admin without password



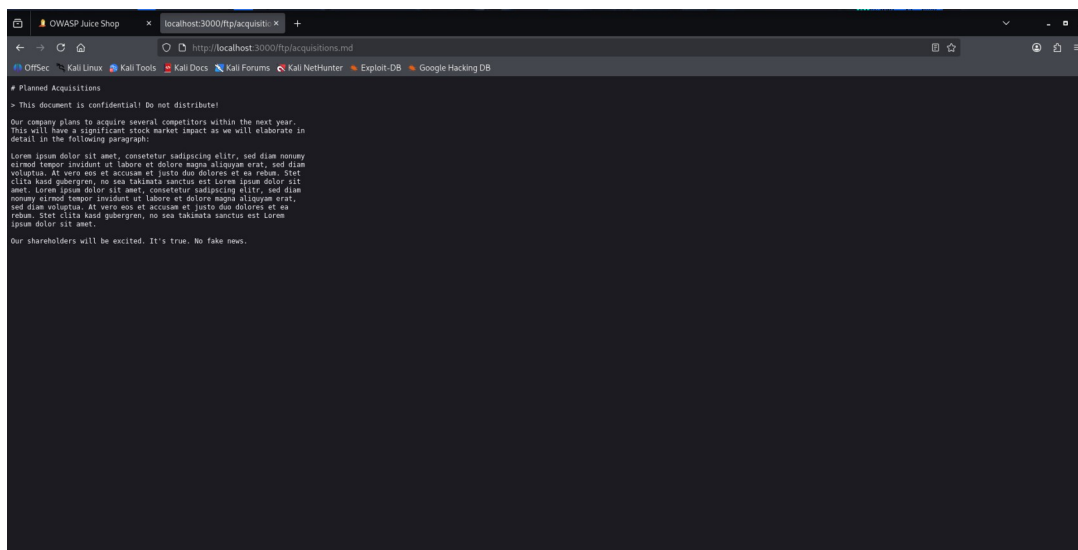
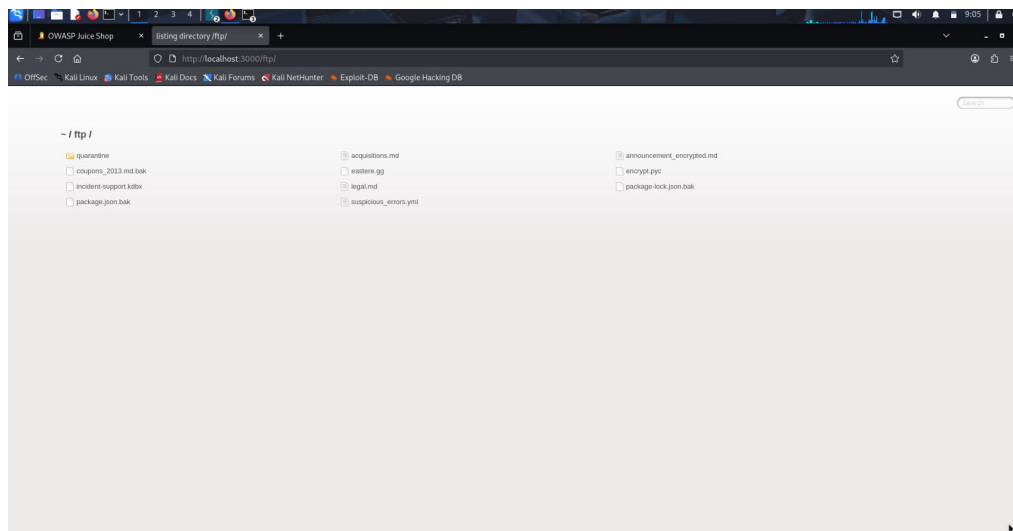
4.2 Cross-Site Scripting (Reflected XSS)

- Payload: `<iframe src="javascript:alert('XSS')"></iframe>`
- Impact: Execution of malicious script in victim's browser



4.3 Exposed /ftp/ Directory (Information Disclosure)

- Steps: Navigate to `http://localhost:3000/ftp/`
- Sensitive files found: `acquisitions.md`, `package.json.bak`, `coupons_2013.md.bak`
- Impact: Attacker can download sensitive internal files



4.4 Hidden Endpoints (Dirsearch)

- Discovered: `/api/config`, `/api/error_log`, `/admin`, `/backup`
- Risk: Potential exposure of configuration and admin panels

```
kali@kali: ~
Session Actions Edit View Help
500 3KB http://localhost:3000/api.log
500 3KB http://localhost:3000/api.py
500 3KB http://localhost:3000/api/_swagger_/
500 3KB http://localhost:3000/api/_swagger_/
500 3KB http://localhost:3000/api/api
500 3KB http://localhost:3000/api/api-docs
500 3KB http://localhost:3000/api/apidocs
500 3KB http://localhost:3000/api/2/issue/createmeta
500 3KB http://localhost:3000/api/application.wadl
500 3KB http://localhost:3000/api/apidocs/swagger.json
500 3KB http://localhost:3000/api/batch
500 3KB http://localhost:3000/api/cask/graphql
500 3KB http://localhost:3000/api/docs/
500 3KB http://localhost:3000/api/jsonws
500 3KB http://localhost:3000/api/error_log
500 3KB http://localhost:3000/api/config
500 3KB http://localhost:3000/api/index.html
500 3KB http://localhost:3000/api/docs
500 3KB http://localhost:3000/api/profile
500 3KB http://localhost:3000/api/jsonws/invoke
500 3KB http://localhost:3000/api/swagger
500 3KB http://localhost:3000/api/login.json
500 3KB http://localhost:3000/api/swagger-ui.html
500 3KB http://localhost:3000/api/spec/swagger.json
500 3KB http://localhost:3000/api/snapshots
500 3KB http://localhost:3000/api/package_search/v4/documentation
500 3KB http://localhost:3000/api/proxy
500 3KB http://localhost:3000/api/swagger.yaml
500 3KB http://localhost:3000/api/swagger.json
500 3KB http://localhost:3000/api/swagger.yml
500 3KB http://localhost:3000/api/swagger/static/index.html
500 3KB http://localhost:3000/api/swagger/swagger
500 3KB http://localhost:3000/api/swagger/index.html
500 3KB http://localhost:3000/api/swagger/ui/index
500 3KB http://localhost:3000/api/v1
500 3KB http://localhost:3000/api/timelion/run
500 3KB http://localhost:3000/api/v1/swagger.json
500 3KB http://localhost:3000/api/v1/
500 3KB http://localhost:3000/api/v2/helpdesk/discover
500 3KB http://localhost:3000/apidocs
500 3KB http://localhost:3000/api/v3

(kali@kali)~$
```

=====

5. Risk Assessment

Finding	Serverity	Likelihood
SQL Injection Login Bypass	Critical	High
Reflected XSS	Medium	Medium
Exposed/ftp/directory	Medium	High
Hidden admin/API endpoints	Low-Medium	Medium

=====

6. Remediation Recommendations

- SQL Injection: Use parameterized queries / prepared statements.
- XSS: Implement output encoding and Content Security Policy (CSP).
- Exposed /ftp/: Disable directory listing, add authentication, move sensitive files out of web root.
- Hidden endpoints: Restrict access via authentication or IP whitelisting.

=====

7. Conclusion

The application has several security weaknesses that could be exploited by an attacker. Immediate action is recommended for SQL injection and exposed directory issues. Regular security assessments should be conducted.

=====

Report prepared by: Saloni Kumari
Date: 08 May 2026